mssql.py Usage Guide

Oct 17, 2026 · @Scott

Intro

I wanted an sql tool for ms-sqlserver to use on Linux. The M$ tool is ok but I wanted something more. I have a very dated tool I rolled long ago in Tcl, but it needed work, so I rolled this with AI-Vibing since I have have no real time to fully write one.

Overview

`mssql.py` is a single-file, `sqlite3`-style command-line client for Microsoft SQL Server and MySQL. It runs ad-hoc queries, scripts and piped SQL, offers an interactive prompt with history and tab completion, formats results six ways, and bulk-imports or exports CSV, JSON and JSONL.

It needs Python 3 and the driver for the server you connect to. Each driver is loaded only when you use it, so install just the one you need.

Connecting to	Flag	Driver to install	Extra requirement
SQL Server, SQL login	(default)	`pip install "pymssql>=2.4"`	On Windows: `pip install "pymssql>=2.4" --only-binary :all:`. Older pymssql returns `datetimeoffset` values as raw bytes and misreads the milliseconds of most `time` / `datetime2` values; the tool warns when it connects through one
SQL Server, Windows login (SSO)	`-E`	`pip install pyodbc`	Microsoft ODBC Driver for SQL Server (18 by default)
MySQL	`--mysql`	`pip install "pymysql[rsa]"`	RSA support for SHA-256 authentication without TLS

Run `python3 mssql.py --help` to see every option without connecting. Interactive mode also uses Python's `readline` module when it is available, for history and tab completion.

Custom `.export --query` calls also need SQLGlot: install it with `python3 -m pip install -r requirements-query.txt`. Table exports and ordinary SQL execution use only the driver dependencies.

Install SQLGlot before running `python3 -m unittest test_mssql.py` to include the query-parser regressions; those tests skip when it is unavailable.

Connecting

Connection settings come from command-line flags, an `--env-file`, or environment variables, in that order of precedence. Keep passwords out of flags when you can: anything on a command line can show up in shell history and process listings.

SQL Server with a SQL login

```
export MSSQL_PASSWORD='...'
python3 mssql.py -H myserver -U sa -d mydb
```

A user name is required (`-U` or `MSSQL_USER`). The port defaults to 1433. The database can also be given as the first positional argument: `python3 mssql.py -H myserver -U sa mydb`.

SQL Server with Windows Integrated Authentication

```
python3 mssql.py -H myserver -E -d mydb
```

`-E` (`--trusted-connection`) signs in as your current OS login through Kerberos/NTLM, so no user or password is sent; combining it with `-U` or `-W` is an error. Use `--odbc-driver` (or `MSSQL_ODBC_DRIVER`) to pick a driver other than "ODBC Driver 18 for SQL Server". Add `--trust-server-certificate` for servers with self-signed TLS certificates, which ODBC Driver 18 otherwise rejects.

MySQL

```
export MYSQL_PASSWORD='...'
python3 mssql.py --mysql -H myserver -U app -d mydb
```

`--mysql` uses the same `-H/-P/-U/-W/-d` flags, but reads `MYSQL_*` environment variables instead of `MSSQL_*` ones and defaults to port 3306. It never falls back to `MSSQL_*` values, so a SQL Server password is never sent to a MySQL server. It cannot be combined with `-E`, `--odbc-driver` or `--trust-server-certificate`.

Reading settings from an env file

```
python3 mssql.py --mysql --env-file .env-mysql
```

`--env-file` reads a dotenv-style `KEY=VALUE` file. It accepts the active backend's own names (`MYSQL_HOST`, `MSSQL_USER`, and so on) and these generic names, mapped to whichever backend you are using:

Generic name	Maps to
`DB_HOST`	host
`DB_PORT`	port
`DB_USER`	user
`DB_PASSWORD`	password
`DB_NAME` or `DB_DATABASE`	database

If a file has both a backend-specific name and its generic alias, the specific one wins; any other keys are ignored. The file may use `#` comments, blank lines, a leading `export`, single- or double-quoted values, and a `#` comment (preceded by whitespace) after an unquoted value. There is no `${VAR}` expansion. The values are kept in memory and never copied into the process environment.

A missing or malformed file stops the tool before it connects (exit code 1). Error messages give only the line number, never the line, so a password can't end up on screen. `.reconnect` reuses the values read at startup.

Precedence

1. Command-line flags (`-H`, `-P`, `-U`, `-W`, `-d`, positional database)
2. Values from `--env-file`
3. Environment variables (`MSSQL_*`, or `MYSQL_*` with `--mysql`)
4. Built-in defaults: host `localhost`, port 1433 (3306 for MySQL)

An explicitly empty password (`--password ''`) overrides a password from the env file or environment. MySQL passwords are sent as UTF-8.

Timeouts

`--timeout` sets the login/connect timeout and `--query-timeout` the per-query timeout; both default to 30 seconds and must be greater than 0.

On SQL Server with pymssql, a query that runs past `--query-timeout` drops the connection: the server stops the query and rolls back any open transaction. The tool then refuses further SQL until you run `.reconnect`, and says so if a transaction was open. With `--mysql` see Limitations.

Running SQL

The tool picks one input source, in this priority order: `-c` text, then `-f` file, then a positional SQL file, then piped stdin, and otherwise the interactive prompt. Giving more than one of `-c`, `-f` and a positional SQL file is an error.

```
# One command
python3 mssql.py -H myserver -U sa -d mydb -c "SELECT TOP 5 * FROM users"

# A script file (-f, or as the second positional argument after DATABASE)
python3 mssql.py -H myserver -U sa -f query.sql
python3 mssql.py -H myserver -U sa mydb query.sql

# Piped input
echo "SELECT 1" | python3 mssql.py -H myserver -U sa
```

With `-d DATABASE`, a single positional argument is read as the SQL file rather than the database.

Batches and GO

Scripts are split into batches on `GO` lines, as in SSMS and sqlcmd. Each batch is sent to the server as one request, so a batch can hold several statements and return several result sets; all of them are printed.

- `GO` must be alone on its line (a trailing `--` comment is allowed) and is matched case-insensitively.
- `GO 5` runs the preceding batch 5 times; the limit is 10,000.
- A `GO` inside a string, quoted identifier or block comment is ignored.
- `GO` also works with `--mysql`, where it is handled by this tool, not the server.
- Line endings are passed through exactly as written, so CR, CRLF and characters such as U+2028 inside string literals reach the server unchanged.

Input limits

Script files and piped input must be UTF-8 without a byte-order mark, and at most 64 MiB. With pymssql, SQL text containing a NUL character (U+0000) is refused, because the driver would silently cut the statement off at it; write `NCHAR(0)` or `CHAR(0)` instead.

Exit codes

Code	Meaning
0	Every batch succeeded
1	A batch failed, the connection failed, or an input or env file couldn't be read
2	Invalid command-line usage (argparse error)

When a batch fails, its error goes to stderr and the remaining batches still run; the exit code is 1 at the end.

Server messages from `PRINT` and `RAISERROR` with severity 10 or lower (including `WITH NOWAIT` progress messages and `Changed database context to ...`) are printed to stderr as they arrive, so they never mix with CSV or JSON results. Higher severities are errors and are reported as such. Writing to a closed pipe (for example `| head`) exits quietly with code 1, after committing any changes that batch already made.

Interactive REPL

With no `-c`, `-f`, script file or piped input, the tool opens an interactive prompt: `mssql>` for SQL Server, `mysql>` with `--mysql`.

```
$ python3 mssql.py --mysql --env-file .env-mysql
MySQL Query Tool  (pymysql 2.2.8)
Connected to: app_user@127.0.0.1:3306/app_database
Type .help for help, .quit to exit. Terminate SQL with ; or GO.

mysql> SELECT id, name
   ...> FROM users
   ...> WHERE active = 1;
```

Running statements

- A statement runs when a line ends with `;`, or when you enter `GO` (optionally `GO n`) on its own line. Until then, the `...>` prompt collects more lines.
- A `;` inside a string, quoted identifier or comment doesn't count, so a multi-line string can contain semicolons.
- Trailing semicolons are preserved, including the terminator required by SQL Server's `MERGE`. Everything typed since the last run goes to the server as one batch, so several statements can run together.
- Dot commands (`.tables`, `.mode csv`, ...) are recognized only at the start of a new statement, never on a continuation line.
- Row counts are shown by default in interactive mode; turn them off with `.rowcount off`.

Keys

Key	Effect
Ctrl-C at the prompt	Discards the statement being typed
Ctrl-C while a query runs	Prints "Interrupted." and returns to the prompt. Re-reads transaction state when the connection survives; if MySQL closes it, attempts cancellation and asks you to `.reconnect`
Ctrl-D (EOF)	Exits, same as `.quit` or `.exit`
Tab	Completes SQL keywords (upper or lower case) and dot commands

History

When `readline` is available, up to 1,000 lines of history are saved to `~/.mssql_query_history` (on Windows, `%APPDATA%\mssql_query_history.txt`) and reloaded the next time you start. SQL Server and MySQL sessions share the same history file.

Output modes and options

Results print as a bordered table by default. Choose another mode with `--mode` on the command line or `.mode` at the prompt.

Mode	Output	NULL shown as	Headers off?	Streams large results?
`table`	Bordered ASCII table, columns sized to fit	`NULL`	Yes	No

Mode	Output	NULL shown as	Headers off?	Streams large results?
`csv`	RFC 4180 CSV	Empty field	Yes	Yes
`json`	Indented JSON array of objects	`null`	No	No
`vertical`	One `column: value` line per field, rows separated by a rule	`NULL`	No	No
`line`	One `col=value, col=value` line per row	`NULL`	No	Yes
`insert`	One `INSERT INTO ... VALUES (...);` per row	`NULL`	No	Yes

Streaming modes write results in 500-row chunks, so very large result sets aren't held in memory first. (pymysql still loads each MySQL result fully on the client before the tool sees it.)

Value formatting

- Binary values appear as `0x`-prefixed hex in CSV and JSON. INSERT output uses `0x...` for SQL Server and `X'...'` for MySQL.
- MySQL `TIME` values print as `[-]HH:MM:SS[.ffffff]`, for example `-30:15:00`.
- In JSON, NaN and infinity become `null`; dates, decimals and other non-JSON types become strings.
- JSON requires unique column labels. Alias duplicate columns in a join, such as `SELECT a.id AS a_id, b.id AS b_id`, before selecting JSON output.
- INSERT output writes datetimes with millisecond precision and a four-digit year, and preserves SQL Server `datetimeoffset` offsets. SQL Server strings containing NUL are written as a `REPLACE(...)` expression, and a backslash before a line break is doubled so T-SQL's line continuation doesn't drop it; both replay exactly. NaN or infinity become `NULL`, booleans become `1`/`0`, and strings use `''` escaping (MySQL also doubles backslashes). SQL Server strings use Unicode literals (`N'...'`). Identifiers are quoted `[like_this]` for SQL Server and `` `like_this` `` for MySQL.

Insert mode

Insert mode needs a target table name, which goes into each generated statement:

```
python3 mssql.py -H myserver -U sa -d mydb --mode insert --insert-table dbo.users_copy -c
"SELECT * FROM dbo.users"
```

At the prompt, use `.mode insert dbo.users_copy`, or `.output FILE --insert TABLE` to switch mode and send output to a file in one step. Row counts and timing messages go to stderr, leaving only the generated SQL in the output file.

Datetime values are written to the millisecond by default, because SQL Server's legacy `datetime` type rejects literals with more fractional digits. Add `--insert-microseconds` to keep sub-millisecond digits from `datetime2` and `datetimeoffset` columns. Values without sub-millisecond digits, which includes everything read from a `datetime` column, are still written to the millisecond, so a table that mixes both types replays exactly. Only the target column type matters: a `datetime` target column rejects a value that carries microseconds. Python datetimes stop at microseconds, so the seventh fractional digit of `datetime2(7)` is lost either way, and pymssql before 2.4 only reads milliseconds.

Other output options

Option	Prompt equivalent	Effect
`--no-headers`	`.headers off`	Omits the header row (table and csv modes)
`--timer`	`.timer on`	Prints `Elapsed: N ms` to stderr after each statement
`--rowcount`	`.rowcount on`	Prints `(N rows)` to stderr after each result; on by default at the prompt
`-o FILE`	`.output FILE`	Writes results to FILE (UTF-8, overwritten) instead of stdout

Results go to stdout or the `-o` file. Status messages, progress and errors always go to stderr, so redirected output holds only data. When row counts are enabled, statements without a result set print `(N rows affected)` to stderr; `.rowcount off` hides these too.

Dot commands

Dot commands work only at the interactive prompt, at the start of a new statement. `.export`, `.import`, `.output` and `.read` split their arguments like a shell command line, so quote paths that contain spaces.

Command	What it does
`.help`	Lists all dot commands
`.databases`	Lists databases. SQL Server: name, state, compatibility level. MySQL: name, default character set, collation

Command	What it does
`.tables [PATTERN]`	Lists tables and views whose name matches `LIKE '%PATTERN%'` (so `_` and `%` are wildcards) or contains PATTERN literally, which finds names with `[`, `%` or `_` in them
`.views [PATTERN]`	Lists views, filtered the same way
`.procs [PATTERN]`	Lists stored procedures and functions, filtered the same way
`.schema TABLE`	Shows each column's name, data type, maximum length, nullability, default and position
`.columns TABLE`	Same as `.schema`
`.export TABLE FILE [OPTS]`	Writes a table or query to CSV, JSON or JSONL (see Exporting data)
`.import FILE TABLE [OPTS]`	Bulk-loads CSV, JSON or JSONL into a table (see Importing data)
`.mode MODE [TABLE]`	Sets the output mode; `insert` mode needs TABLE
`.headers on|off`	Shows or hides header rows
`.output FILE [--append] [--insert TABLE]`	Sends results to FILE; `.output -` returns to stdout
`.read FILE`	Runs a SQL script, split on `GO` like `-f`
`.reconnect`	Reconnects with the original settings and resets transaction tracking
`.timer on|off`	Shows or hides elapsed time per statement
`.rowcount on|off`	Shows or hides row counts
`.quit` / `.exit`	Exits

Table names

- **SQL Server:** `table`, `schema.table` or `database.schema.table`, with optional `[brackets]`. If a bare name exists in several schemas, `.schema` asks you to qualify it.
- **MySQL:** `table` (in the current database) or `database.table`, with optional `` `backticks` ``.
- **Listings on MySQL:** `.tables`, `.views` and `.procs` show only the current database. When no database is selected, they show every database except `mysql`, `sys`, `performance_schema` and `information_schema`.

.output details

`.output FILE` overwrites FILE; add `--append` to append instead. The previous output file is closed when you switch, and a failure while closing it is reported, because it can mean that file is incomplete. `--insert TABLE` also switches to insert mode for TABLE.

Reopening the same file flushes the previous writer before overwriting or appending. If that flush fails, the current handle is retained and the file is not truncated. A write failure returns to the prompt so you can select another destination with `.output FILE` or `.output -`; it leaves your explicit transaction open. A failure flushing the final output file at exit produces a nonzero exit status.

.reconnect

Use `.reconnect` after a dropped connection or a query timeout. It opens a new connection before closing the old one, so if reconnecting fails you keep the old session. Any transaction open on the old connection is lost.

Exporting data

`.export` streams a whole table, or the result of one read-only query, to a CSV, JSON or JSONL file, or to stdout.

```
.export dbo.users users.csv
.export dbo.users active.jsonl --where "active = 1" --columns id,email --order-by id
.export - report.json --query "SELECT region, SUM(total) AS total FROM sales GROUP BY region"
.export dbo.users - --format csv --delimiter ";"
```

Option	Default	Effect
`--format csv|json|jsonl`	From FILE's extension (`.csv`, `.json`, `.jsonl`)	Output format; required when FILE is `-` or has another extension
`--query SQL`	None	Exports a custom query instead of a table; use `-` as TABLE
`--where CONDITION`	None	Adds a `WHERE` clause (table export only)
`--columns a,b,...`	All columns	Exports only these columns, quoted for you (table export only); bracket a name that contains a comma, e.g. `[a,b]`
`--order-by EXPR`	None	Adds an `ORDER BY` clause (table export only)

Option	Default	Effect
`--delimiter CHAR`	`,`	CSV field separator; exactly one character
`--no-header`	Off	Leaves out the CSV header row
`--null-as STR`	Empty (CSV), `null` (JSON)	Text written for NULL
`--chunk N`	500	Rows fetched per round trip
`--append`	Off	Appends to FILE instead of replacing it; not allowed for JSON array output

Use `-` or `stdout` as FILE to write to standard output.

Formats

- **CSV** writes one header row (unless `--no-header`, or when appending to a non-empty file) and one line per row. Binary values are hex-encoded as `0x...`. An empty string is written as `""` and NULL as an empty unquoted field, so `.import` can tell them apart; a row of a one-column export that holds NULL is therefore a blank line.
- **JSON** writes a single array of objects, one per row.
- **JSONL** writes one JSON object per line, which suits appending and very large exports.

Safety rules

- `--query` needs SQLGlot and must parse as a single read-only `SELECT` or `WITH` query. Extra statements are rejected even without semicolons. Statements that change data, `SELECT ... INTO`, and MySQL executable comments are rejected. Embedded semicolons are rejected; trailing semicolons are allowed.
- JSON and JSONL require unique column labels; use aliases for duplicate columns.
- `--where` and `--order-by` are pasted into the SQL as written; only `;` is rejected. Never pass them untrusted input.
- A bare table name means `dbo.table` on SQL Server, and the current database on MySQL.

Failures and partial files

A new export is written to a temporary file next to the target and renamed over it only after it completes. If it fails or you press Ctrl-C, the target is left untouched. With `--append`, a failed export truncates the file back to its previous length. Progress and the final `Exported N rows ... in Xs` summary go to stderr.

Importing data

`.import` bulk-loads a CSV, JSON or JSONL file into an existing table using parameterized, batched `INSERT`s, committing after each batch. On SQL Server each batch is sent as requests of up to 1,000 single-row `INSERT`s (fewer for wide rows or large values), so a request costs one round trip instead of one per row; each row still converts to the column types on its own, exactly as a lone `INSERT` would. On MySQL, pymysql combines the rows of a batch itself.

```
.import users.csv dbo.users
.import users.csv dbo.users --truncate --batch 5000
.import export.jsonl staging.events --on-error skip
.import data.txt dbo.users --format csv --delimiter "|" --no-header --columns id,name,email
```

Option	Default	Effect
`--format csv|json|jsonl`	From FILE's extension	Input format; required for other extensions
`--batch N`	1000	Rows per batch; each batch is committed on its own
`--skip N`	0	Skips the first N data rows (after any CSV header)
`--truncate`	Off	Empties the table first: `TRUNCATE TABLE` on SQL Server, `DELETE FROM` on MySQL
`--no-header`	Off	The CSV has no header row
`--delimiter CHAR`	`,`	CSV field separator
`--columns a,b,...`	From the file	Explicit target column list
`--on-error skip|abort`	`abort`	What to do when a batch fails
`--keep-identity`	Off	Inserts the file's values into a SQL Server `IDENTITY` column (`SET IDENTITY_INSERT`); without it, a file that includes the identity column is refused before anything is inserted

How columns are matched

- **CSV with a header:** header names are matched to table columns case-insensitively. Duplicate header names are an error. `--columns` picks a subset of the header to load.

- **CSV without a header (`--no-header`):** fields map in order to the `--columns` list or, without it, to every table column in table order. Each row must have exactly that many fields.
- **JSON / JSONL:** each record must be an object. Columns come from the keys of the first record imported, or from `--columns`. Without `--columns`, the tool warns once if a later record has keys the first one lacks, since those values are not imported.
- **Generated columns** (SQL Server `rowversion` / `timestamp` and computed columns; MySQL generated columns) are skipped automatically, with a note, because the server computes them; a full-table export therefore re-imports into the same table definition. Naming one in `--columns` sends it anyway.
- **Identity columns:** add `--keep-identity` to keep the file's values (for example when copying a table), or leave the column out with `--columns` to let SQL Server assign new ones. MySQL accepts explicit `AUTO_INCREMENT` values without the option.
- An empty, unquoted CSV field becomes NULL. A quoted empty field (`""`) is an empty string for text columns and NULL for any other column type. In a one-column CSV a blank line is a NULL row. Every column you name must exist in the table, or the import stops before inserting anything.
- An empty JSON array or JSONL file (such as the export of an empty table) imports no rows.
- For binary columns (`binary`, `varbinary`, `image`, or MySQL blob types) and SQL Server's `geography`, `geometry` and `hierarchyid` columns, exported `0x...` values are decoded back to bytes, including `0x` for empty bytes. Other values in those columns (such as WKT text like `POINT(1 2)`) are passed through as text. Text columns retain their literal values. Malformed binary hex follows `--on-error abort|skip`.

A JSON array file is read into memory and is limited to 512 MiB; JSONL is streamed line by line, so use JSONL for large imports. A single CSV field may hold up to 512 Mi characters. A bare table name means `dbo.table` on SQL Server and the current database on MySQL.

Errors

- **`--on-error abort`** (default): the failing batch is rolled back and the import stops. Batches committed earlier stay in the table.
- **`--on-error skip`:** the failing batch is rolled back and retried in halves: a half that inserts cleanly is committed at once, a half that fails is split again, so only the bad rows end up retried on their own. They are skipped and reported by row number, and the other rows keep their order. Rows that fail validation (wrong field count, a non-object JSON record) are skipped the same way.
- The summary on stderr reports rows imported, elapsed time, rate, and rows skipped.

- CSV parsing and invalid UTF-8 errors stop the import with a read-error message and close the input. Previously committed batches remain; an explicit transaction remains available for your `COMMIT` or `ROLLBACK`.

--truncate

The table is emptied in the same transaction as the first batch, so if that batch fails the original rows come back. The table is emptied even when the file contributes no rows. On MySQL `DELETE FROM` is used because `TRUNCATE TABLE` commits implicitly there; unlike `TRUNCATE`, it doesn't reset `AUTO_INCREMENT`. Because `DELETE` would apply other tables' `ON DELETE CASCADE` or `SET NULL` rules, the import refuses `--truncate` on a MySQL table that another table's foreign key references (unless `foreign_key_checks` is off), just as `TRUNCATE TABLE` refuses on both servers.

Inside an open transaction

If you run `.import` after `BEGIN TRANSACTION` (SQL Server) or `START TRANSACTION` (MySQL), the import joins your transaction and never commits on its own. You decide whether to `COMMIT` or `ROLLBACK` afterwards. In this case `--on-error skip` stops at the first failing batch instead of retrying, because the transaction may no longer be safe to continue.

Transactions

On both servers, a change you make outside an explicit transaction is committed right away, and a change made inside one waits for your `COMMIT` or `ROLLBACK`. The mechanics differ.

	SQL Server	MySQL (`--mysql`)
Committing ordinary changes	The tool commits after each batch that changes data (`INSERT`, `UPDATE`, `DELETE`, `MERGE`, `EXEC`, or any statement without a result set)	The server auto-commits each statement, as the `mysql` client does
Opening a transaction	`BEGIN TRAN[SACTION] [name]`	`START TRANSACTION` or `BEGIN`
How open transactions are detected	After SQL containing `BEGIN` / `COMMIT` / `ROLLBACK` / `SAVE TRAN`, the tool reads `XACT_STATE()` and `@@TRANCOUNT` from the server, including for skipped conditional controls and named rollbacks	The server's own in-transaction status flag, after every statement
Nesting	Tracked by depth (like `@@TRANCOUNT`); only the outermost `COMMIT` commits	Not supported by MySQL; `BEGIN` inside a transaction commits the first one

	SQL Server	MySQL (`--mysql`)
Savepoints	`SAVE TRAN name` / `ROLLBACK TRAN name`	`SAVEPOINT name` / `ROLLBACK TO SAVEPOINT name` / `RELEASE SAVEPOINT name`
DDL (`CREATE` , `ALTER` , `DROP`)	Transactional	Commits any open transaction implicitly; the tool notices

SQL Server connections start in autocommit mode with both pymssql and pyodbc. Ordinary SQL batches and imports temporarily use a client-owned transaction, then return to autocommit. Explicit transaction controls run without a hidden driver transaction; `BEGIN TRAN` , `COMMIT` , `ROLLBACK` and `SAVE TRAN` are recognized anywhere in a batch, including after another statement on the same line without a semicolon. If the tool cannot verify the server state after a transaction control, it blocks further database work until `.reconnect` . The verification query changes SQL Server session diagnostics such as `@@ROWCOUNT` after transaction controls.

Some batches run in plain autocommit, without the client-owned transaction (and so without the all-or-nothing rollback described below): those containing statements SQL Server refuses inside a transaction (`CREATE` / `ALTER` / `DROP DATABASE` , `BACKUP` , `RESTORE` , `RECONFIGURE` , full-text catalog DDL, `DBCC SHRINKDATABASE` / `SHRINKFILE` , and similar), and those that read or change the session's transaction state (`@@TRANCOUNT` , `XACT_STATE()` , `SET IMPLICIT_TRANSACTIONS` , `SET ANSI_DEFAULTS`). So `SELECT @@TRANCOUNT` shows your own transaction depth.

With `SET IMPLICIT_TRANSACTIONS ON` (or `ANSI_DEFAULTS ON`) the tool behaves like sqlcmd: it neither commits for you nor uses its own transaction, so changes stay pending until you `COMMIT` (imports included), and it re-reads `@@TRANCOUNT` after every batch.

When a statement fails

- **No transaction open:** on SQL Server the tool rolls back whatever the failed batch had done, so nothing half-finished is left behind. On MySQL the server auto-commits each statement as it finishes, so statements in the batch that completed before the failing one stay committed; the tool prints a warning when that happens. Wrap a MySQL batch in `START TRANSACTION` … `COMMIT` if it must be all-or-nothing.
- **Inside your transaction:** the tool leaves it to you and re-reads the real state from the server. On SQL Server that is `XACT_STATE()` and `@@TRANCOUNT` ; on MySQL, the status flag. A deadlock victim's transaction, for example, is then correctly shown as closed.

- **If the rollback itself fails** (usually a dropped connection), the tool says so; run `.reconnect` before continuing.

Leaving with a transaction open

Exiting, or losing the connection, while a transaction is open means the server rolls it back. Commit before `.quit` if you want to keep the changes. `.reconnect` also discards any open transaction.

Reference

```
python3 mssql.py [OPTIONS] [DATABASE] [SQL_FILE]
```

Command-line options

Option	Default	Description
`-H`, `--host HOST`	`localhost`	Server hostname
`-P`, `--port PORT`	1433 (3306 with `--mysql`)	Server port
`-U`, `--user USER`	None	Login name; required except with `-E`
`-W`, `--password PASSWORD`	Empty	Login password; prefer the environment or an env file
`-d`, `--database DATABASE`	Server default	Database to use
`--env-file FILE`	None	Reads connection settings from a dotenv file
`--timeout SECS`	30	Login/connect timeout
`--query-timeout SECS`	30	Per-query timeout
`--mysql`	Off	Connects to MySQL with pymysql
`-E`, `--trusted-connection`	Off	Windows Integrated Authentication via pyodbc
`--odbc-driver DRIVER`	ODBC Driver 18 for SQL Server	ODBC driver for `-E`
`--trust-server-certificate`	Off	Skips TLS certificate validation for `-E`

Option	Default	Description
`-c`, `--command SQL`	None	Runs SQL and exits
`-f`, `--file FILE`	None	Runs a SQL file and exits
`DATABASE` (positional)	None	Database name, if `-d` isn't used
`SQL_FILE` (positional)	None	SQL file to run after DATABASE
`--mode MODE`	`table`	`table`, `csv`, `json`, `vertical`, `line` or `insert`
`--insert-table TABLE`	None	Target table for `--mode insert` (required with it)
`--insert-microseconds`	Off	In insert mode, keep sub-millisecond digits of datetime values (see Insert mode)
`--no-headers`	Off	Suppresses column headers
`--timer`	Off	Shows execution time per statement
`--rowcount`	Off (on at the prompt)	Shows row counts
`-o`, `--output FILE`	stdout	Writes results to FILE
`-h`, `--help`		Shows help and exits

Environment variables

SQL Server	MySQL (`--mysql`)	Generic `--env-file` name	Meaning
`MSSQL_HOST`	`MYSQL_HOST`	`DB_HOST`	Server hostname
`MSSQL_PORT`	`MYSQL_PORT`	`DB_PORT`	Server port
`MSSQL_USER`	`MYSQL_USER`	`DB_USER`	Login name
`MSSQL_PASSWORD`	`MYSQL_PASSWORD`	`DB_PASSWORD`	Login password
`MSSQL_DATABASE`	`MYSQL_DATABASE`	`DB_NAME`, `DB_DATABASE`	Database
`MSSQL_ODBC_DRIVER`			ODBC driver name for `-E`

The generic names are read only from an `--env-file`, never from the process environment.

MySQL notes, limitations and troubleshooting

How MySQL SQL is read

With `--mysql`, the tool reads your SQL by MySQL's rules when deciding where statements end, where `GO` lines are, and what is a string or a comment:

- A backslash escapes the next character inside `'...'` and `"..."` strings, so `'it\'s'` is one string.
- `#` starts a comment, and so does `--` followed by whitespace; `1--1` is arithmetic.
- `` `backticks` `` quote identifiers; brackets have no special meaning.
- Block comments don't nest: the first `*/` closes the comment.

Generated INSERT statements double backslashes and assume the server's default `sql_mode`. If your server enables `NO_BACKSLASH_ESCAPES`, those statements will store doubled backslashes.

Limitations

- **`sql_variant` columns with pymssql.** pymssql returns `sql_variant` values as raw bytes without their base type, and some mixes of base types make it drop the connection. Cast such columns to a concrete type in your query (`CAST(col AS nvarchar(4000))`, or `SQL_VARIANT_PROPERTY(col, 'BaseType')` to see the type).
- **Large insert-mode dumps.** `--mode insert` output has no `GO` lines, so piping it back runs as a single batch: raise `--query-timeout` for big dumps, and keep each script under the 64 MiB input limit.
- **No `DELIMITER` command.** At the prompt, a `CREATE PROCEDURE` or trigger body containing `;` runs as soon as a line ends with `;`, on either server. Put such definitions in a file and run it with `-f` or `.read`; the whole file between `GO` lines is sent as one batch.
- **MySQL query timeouts drop the connection.** `--query-timeout` is pymysql's socket read timeout, so a query that runs too long closes the connection. The server would otherwise keep running the statement (and, under autocommit, commit it), so the tool opens a second connection and sends `KILL QUERY` to stop it; if that fails it warns that the statement may still complete. Run `.reconnect` afterwards, and raise `--query-timeout` for long jobs.
- **MySQL results are loaded fully into memory.** pymysql's default cursor buffers each result set on the client, so a very large `.export` or CSV output on MySQL uses memory in proportion to the result.
- **Binary in table mode.** Table, vertical and line modes print binary values as Python bytes (`b'\x00\xff'`); use csv, json or insert mode for hex output.

Troubleshooting

Message	Cause and fix
`pymssql is not installed` / `pyodbc is not installed` / `pymysql is not installed`	Install the driver for the backend you chose (see Overview)
`no user specified (--user or MSSQL_USER)`	Pass `-U`, set the variable, or add `DB_USER` to your env file. With `--mysql` the variable is `MYSQL_USER`; `MSSQL_USER` is never used
`invalid MSSQL_PORT '...'` / `invalid MYSQL_PORT '...'`	The port value isn't a whole number
`Error reading env file: FILE:N: expected KEY=VALUE`	Line N of the env file isn't `KEY=VALUE`; the line itself isn't printed in case it holds a secret
`Connection failed: ...`	The server rejected or couldn't be reached; check host, port, credentials and firewall. For `-E` with a self-signed certificate, add `--trust-server-certificate`
`Rollback failed after ... Reconnect before issuing more SQL.`	The connection is probably gone; run `.reconnect`
`Cannot detect format from '...'`	Add `--format csv|json|jsonl` to `.export` or `.import`
`Table name '...' is ambiguous across schemas`	Qualify the name as `schema.table`